Experiment No.08

Aim:Train a basic Federated Learning model using TensorFlow Federated on small datasets stored on different devices.

Name: Fulari Tejasvini
Roll No.: EN24107039 Batch: B

In [6]:
```
import sys
import numpy as np
import tensorflow as tf
import tensorflow_federated    as tff

print("Python:", sys.version.split()[0])
print("NumPy:", np.__version__)
print("TensorFlow:", tf.__version__)
print("TensorFlow Federated:", tff.__version__)
```

```
Python: 3.9.13
NumPy: 1.25.2
TensorFlow: 2.14.1
TensorFlow Federated: 0.87.0
```

In [2]:
```
train_data, _ = tff.simulation.datasets.emnist.load_data(
    only_digits=True
)

client_ids = train_data.client_ids[:3]

print("Devices:", client_ids)
```

```
Devices: ['f0000_14', 'f0001_41', 'f0005_26']
```

In [3]:
```
def preprocess(ds):
    return ds.map(
        lambda x: (
            tf.image.convert_image_dtype(x["pixels"], tf.float32),
            x["label"]
        )
    ).batch(20)

federated_data = [
    preprocess(
        train_data.create_tf_dataset_for_client(cid)
    )
    for cid in client_ids
]

print("Data prepared for 3 devices.")
```

```
Data prepared for 3 devices.
```

In [4]:
```
def model_fn():
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(28, 28)),
        tf.keras.layers.Flatten(),
```

```
        tf.keras.layers.Dense(10, activation="softmax")
    ])

    return tff.learning.models.from_keras_model(
        model,
        input_spec=federated_data[0].element_spec,
        loss=tf.keras.losses.SparseCategoricalCrossentropy(),
        metrics=[tf.keras.metrics.SparseCategoricalAccuracy()]
    )

learning_process = tff.learning.algorithms.build_weighted_fed_avg(
    model_fn=model_fn,
    client_optimizer_fn=tff.learning.optimizers.build_sgdm(0.1),
    server_optimizer_fn=tff.learning.optimizers.build_sgdm(1.0)
)

state = learning_process.initialize()

print("Federated Learning model initialized.")
```

```
2026-09-29 10:39:36.662894: I tensorflow/core/grappler/devices.cc:66] Number o
f eligible GPUs (core count >= 8, compute capability >= 0.0): 0
2026-09-29 10:39:36.663098: I tensorflow/core/grappler/clusters/single_machin
e.cc:361] Starting new session
2026-09-29 10:39:36.684261: I tensorflow/core/grappler/devices.cc:66] Number o
f eligible GPUs (core count >= 8, compute capability >= 0.0): 0
2026-09-29 10:39:36.684392: I tensorflow/core/grappler/clusters/single_machin
e.cc:361] Starting new session
Federated Learning model initialized.
```

In [7]:

```
for round_num in range(1, 3):

    result = learning_process.next(
        state,
        federated_data
    )

    state = result.state

    print(f"Round {round_num} completed")
    print(result.metrics)

print("\nFederated Learning completed!")
```

```
Round 1 completed
OrderedDict([('distributor', ()), ('client_work', OrderedDict([('train', Order
edDict([('sparse_categorical_accuracy', 0.10181818), ('loss', 14.47698), ('num
_examples', 275), ('num_batches', 15)]))])), ('aggregator', OrderedDict([('mea
n_value', ()), ('mean_weight', ())])), ('finalizer', OrderedDict([('update_non
_finite', 0)]))])
Round 2 completed
OrderedDict([('distributor', ()), ('client_work', OrderedDict([('train', Order
edDict([('sparse_categorical_accuracy', 0.10181818), ('loss', 14.47698), ('num
_examples', 275), ('num_batches', 15)]))])), ('aggregator', OrderedDict([('mea
n_value', ()), ('mean_weight', ())])), ('finalizer', OrderedDict([('update_non
_finite', 0)]))])

Federated Learning completed!
```

In [ ]:

In [ ]: